Experiment 11: Write a Python program to Map Coloring Using CSP

Aim:

To write a Python program to implement the Map Coloring problem using a Constraint Satisfaction Problem (CSP).

Theory:

A Constraint Satisfaction Problem consists of three components:

1. Variables: The regions of the map (A, B, C, D).
2. Domains: The available colors (Red, Green, Blue).
3. Constraints: Neighboring regions must have different colors.

The backtracking algorithm assigns colors to regions and checks whether the constraints are satisfied.

Algorithm:

1. Start the program.
2. Define the regions and their neighboring regions.
3. Define the available colors.
4. Select an uncolored region.
5. Assign a color if it does not conflict with its neighbors.
6. If a conflict occurs, try another color using backtracking.
7. Print the solution when all regions are colored.
8. Stop the program.

Python Program:

```
colors = ["Red", "Green", "Blue"]

graph = {

   "A": ["B", "C"],

   "B": ["A", "C", "D"],

   "C": ["A", "B", "D"],

   "D": ["B", "C"]

}

coloring = {}
```

```
def is_safe(region, color):
    for neighbor in graph[region]:
        if neighbor in coloring and coloring[neighbor] == color:
            return False
    return True
def solve():
    if len(coloring) == len(graph):
        return True
    region = next(r for r in graph if r not in coloring)
    for color in colors:
        if is_safe(region, color):
            coloring[region] = color
            if solve():
                return True
            del coloring[region]
    return False
if solve():
    print("Map Coloring Solution:")
    for region, color in coloring.items():
        print(region, ":", color)
else:
    print("No solution exists")
```

exp 11 csp.py - C:/Users/yadal/OneDrive/Documents/AI/exp 11 csp.py (3.14.7)

File Edit Format Run Options Window Help

```
colors = ["Red", "Green", "Blue"]

graph = {
    "A": ["B", "C"],
    "B": ["A", "C", "D"],
    "C": ["A", "B", "D"],
    "D": ["B", "C"]
}

coloring = {}

def is_safe(region, color):
    for neighbor in graph[region]:
        if neighbor in coloring and coloring[neighbor] == color:
            return False
    return True

def solve():
    if len(coloring) == len(graph):
        return True

    region = next(r for r in graph if r not in coloring)

    for color in colors:
        if is_safe(region, color):
            coloring[region] = color

            if solve():
                return True

            del coloring[region]

    return False

if solve():
    print("Map Coloring Solution:")
    for region, color in coloring.items():
        print(region, ":", color)
else:
```

Output:

```
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:/Users/yadal/OneDrive/Documents/AI/exp 11 csp.py ========
Map Coloring Solution:
A : Red
B : Green
C : Blue
D : Red
```

Result:

Thus, the Map Coloring problem was successfully implemented using CSP and the backtracking algorithm in Python.